

หัวข้อโครงการ	AI ดันเจี้ยนมาสเตอร์สำหรับการเล่นเทเบิลท็อป RPG แบบแคชวล
หน่วยกิต	6
ผู้เขียน	นายภนลภัส สุทธิมาลา
อาจารย์ที่ปรึกษา	ผศ.ดร. สุริยา นักสุภักพงศ์ ดร. ไพสิฐ ชันอาสา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
คณะ	สถาบันวิทยาการหุ่นยนต์ภาคสนาม
ปีการศึกษา	2568

บทคัดย่อ

งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนาต้นแบบระบบปัญญาประดิษฐ์ที่ทำหน้าที่เป็น Dungeon Master (AI DM) สำหรับการเล่นเกม Tabletop RPG ในกลุ่มผู้เล่นแคชวล เพื่อลดข้อจำกัดในการที่ต้องพึ่งพา Dungeon Master ที่เป็นมนุษย์ซึ่งต้องอาศัยทั้งความรู้ความเข้าใจในกฎกติกาและการเตรียมตัวล่วงหน้าเป็นอย่างมาก ระบบที่พัฒนาขึ้นจะมุ่งเน้นการสร้างประสบการณ์การเล่นที่เข้าถึงง่ายและเป็นธรรมชาติ โดยจะประกอบด้วยกระบวนการหลักคือ การรับคำสั่งเสียง (Voice Command) จากผู้เล่นผ่านระบบการรู้จำเสียงพูด (ASR) และนำไปประมวลผลร่วมกับแบบจำลองภาษาขนาดใหญ่ (LLM) เพื่อสร้างสรรค์เนื้อเรื่องและสถานการณ์ในเกม ระบบดังกล่าวจะถูกกำกับด้วย Rules Engine ที่ออกแบบตามกติกา “Lite 5e” เพื่อรักษาความสมดุลและความถูกต้องของกลไกเกม พร้อมทั้งมีระบบ Game State Tracker สำหรับติดตามสถานะต่างๆ ของผู้เล่นและเกมอย่างต่อเนื่อง เช่น ค่าพลังชีวิต ไอเทม และตำแหน่ง นอกจากนี้ ระบบจะแสดงผลผ่านส่วนต่อประสานกับผู้ใช้ (UI) ที่เรียบง่ายซึ่งออกแบบมาเพื่อการใช้งานบนอุปกรณ์เดียว

คำสำคัญ : เอไอ ดันเจี้ยนมาสเตอร์ / เทเบิลท็อปอาร์พีจี / แบบจำลองภาษาขนาดใหญ่ (LLM) / การรู้จำเสียงพูดอัตโนมัติ (ASR) / เอ็นจินกฎกติกา / ระบบติดตามสถานะเกม / การประมวลผลภาษาธรรมชาติ

Project Title	AI Dungeon Master for Casual Tabletop RPG Play
Project Credits	6
Candidate	Mr. Pollapaat Suttimala
Project Advisor	Asst.Prof.Dr. Suriya Natsupakpong Dr. Paisit Khanarsa
Program	Bachelor of Engineering
Field of Study	Robotics and Automation Engineering
Faculty	Institute of Field Robotics
Academic Year	2025

Abstract

This project aims to develop a prototype Artificial Intelligence system that functions as a Dungeon Master (AI DM) for casual Tabletop RPG players. The primary goal is to mitigate the constraints of relying on a human Dungeon Master, which requires extensive rule knowledge and significant preparation time. The developed system focuses on creating an accessible and natural gameplay experience. The core process involves receiving voice commands from players via an Automatic Speech Recognition (ASR) system, which are then processed in conjunction with a Large Language Model (LLM) to generate narrative content and in-game scenarios. This system is governed by a Rules Engine designed according to "Lite 5e" rules to maintain game balance and mechanical accuracy. Additionally, a Game State Tracker continuously monitors various player and game statuses, such as hit points, items, and location. Finally, the system presents information through a simple User Interface (UI) designed for single-device usability.

Keywords : AI Dungeon Master / Tabletop RPG / Large Language Model (LLM) / Automatic Speech Recognition (ASR) / Rules Engine / Game State Tracker / Natural Language Processing

บทที่ 1 บทนำ

1.1 ความสำคัญและที่มาของงานวิจัย

ในช่วงไม่กี่ปีที่ผ่านมา กระแสความนิยมในเกมกระดาน (Board Game) และเกมสวมบทบาทบนโต๊ะ (Tabletop Role-Playing Game หรือ Tabletop RPG) ได้รับความสนใจเพิ่มขึ้นอย่างกว้างขวาง โดยเฉพาะเกมอย่าง Dungeons & Dragons (D&D) ซึ่งเป็นเกมที่ผู้เล่นกลุ่มหนึ่งจะสวมบทบาทเป็นตัวละครนักผจญภัย และดำเนินเรื่องราวตามจินตนาการร่วมกัน โดยมีผู้เล่นอีกคนหนึ่งทำหน้าที่เป็น "ดันเจียนมาสเตอร์" (Dungeon Master หรือ DM) คอยสร้างโลก บรรยายสถานการณ์ และควบคุมตัวละครอื่นๆ การรับรู้ที่เพิ่มขึ้นผ่านสื่อออนไลน์และผู้มีอิทธิพลในวงการเกม ได้ดึงดูดให้มีผู้เล่นหน้าใหม่และกลุ่มผู้เล่นทั่วไป (Casual Player) จำนวนมากที่ต้องการสัมผัสประสบการณ์การเล่าเรื่องและการผจญภัยอันเป็นเอกลักษณ์ของเกมประเภทนี้

อย่างไรก็ตาม การเข้าถึงประสบการณ์การเล่น Tabletop RPG ยังคงมีอุปสรรคสำคัญอยู่ที่บทบาทของดันเจียนมาสเตอร์ ซึ่งเปรียบเสมือนหัวใจหลักของเกม การทำหน้าที่ DM ให้สำเร็จลุล่วงได้นั้นต้องอาศัยทักษะและความทุ่มเทหลายด้าน ไม่ว่าจะเป็นความเข้าใจในกฎกติกาที่ซับซ้อน, ความคิดสร้างสรรค์ในการค้นสดและปรับเปลี่ยนเนื้อเรื่องตามการกระทำของผู้เล่น, และที่สำคัญที่สุดคือเวลาในการเตรียมการล่วงหน้าเป็นอย่างมาก ตั้งแต่การสร้างแผนที่ การออกแบบสถานการณ์ ไปจนถึงการเตรียมข้อมูลของศัตรูและตัวละครต่างๆ ซึ่งอาจใช้เวลาเตรียมการหลายวันสำหรับเซสชันการเล่นเพียงครั้งเดียว ข้อจำกัดเหล่านี้ทำให้การเล่นในลักษณะที่ไม่ได้นัดหมายล่วงหน้า หรือการเล่นในบรรยากาศสบายๆ เช่น ที่ร้านกาแฟหรือในงานสังสรรค์ ซึ่งเป็นที่ต้องการของผู้เล่นกลุ่มแคชวล เป็นไปได้ยากอย่างยิ่ง

แม้ว่าในปัจจุบันจะมีเครื่องมือที่ใช้ปัญญาประดิษฐ์ (AI) เข้ามาเป็นทางเลือก เช่น แอปพลิเคชันช่วยเล่าเรื่องอย่าง AI Dungeon หรือการสนทนากับแชทบอททั่วไป แต่เครื่องมือเหล่านี้ยังเป็นเพียงผู้ช่วยเล่าเรื่องที่เน้นการสร้างบทสนทนาโต้ตอบเท่านั้น และยังไม่สามารถมอบประสบการณ์การเล่น "เกม" ได้อย่างสมบูรณ์ เนื่องจากขาดองค์ประกอบหลักที่สำคัญ ได้แก่ การบังคับใช้กฎกติกาเพื่อรักษาความสมดุลและความท้าทาย, กลไกการทอยลูกเต๋าเพื่อวัดผลการกระทำ, และระบบติดตามสถานะของเกม (Game State) ที่ชัดเจน เช่น ค่าพลังชีวิต ไอเทม และตำแหน่งของตัวละคร ส่งผลให้ผลลัพธ์ที่ได้ใกล้เคียงกับการสนทนาโต้ตอบทั่วไปมากกว่าการเล่นเกม นอกจากนี้ ระบบดังกล่าวยังมีข้อจำกัดด้านการจดจำบริบทและความต่อเนื่องของเนื้อเรื่องในระยะยาว (Long-term Memory) ทำให้ไม่สามารถรักษาสถานะของเกมได้อย่างน่าเชื่อถือ

ดังนั้น โครงการนี้จึงมีเป้าหมายในการพัฒนาต้นแบบระบบ "ปัญญาประดิษฐ์ดันเจี้ยนมาสเตอร์" (AI Dungeon Master) สำหรับผู้เล่นกลุ่มแคชชวลโดยเฉพาะ เพื่อลดช่องว่างและอุปสรรคดังกล่าว โดยเป็นการบูรณาการเทคโนโลยีหลายส่วนเข้าด้วยกันเพื่อเอาชนะข้อจำกัดของเครื่องมือที่มีอยู่เดิม ระบบจะผสานการทำงานของระบบการรู้จำเสียงพูด (ASR) เข้ากับแบบจำลองภาษาขนาดใหญ่ (LLM) เพื่อให้ผู้เล่นสามารถโต้ตอบกับเกมได้อย่างเป็นธรรมชาติผ่านคำสั่งเสียง ทำงานร่วมกับ "เอ็นจินกฎกติกา" (Rules Engine) ที่สร้างขึ้นตามกฎ "Lite 5e" เพื่อควบคุมความถูกต้องและความสมดุลของเกม รวมถึงมี "ระบบติดตามสถานะเกม" (Game State Tracker) เพื่อจัดการข้อมูลสำคัญอย่างต่อเนื่อง และนำเสนอผ่านส่วนต่อประสานกับผู้ใช้ (UI) ที่เรียบง่าย เป้าหมายสูงสุดของโครงการคือการลดการพึ่งพา Dungeon Master ที่เป็นมนุษย์ แต่ยังคงรักษาแก่นแท้ของความสมดุล ความท้าทาย และความสนุกของการเล่น Tabletop RPG ไว้ได้อย่างครบถ้วน

1.2 วัตถุประสงค์ของงานวิจัย

1. เพื่อพัฒนาต้นแบบระบบ AI Dungeon Master ที่ผู้เล่นสามารถโต้ตอบผ่านคำสั่งเสียง (ASR) และให้แบบจำลองภาษาขนาดใหญ่ (LLM) ดำเนินเรื่องราวและเหตุการณ์ภายใต้การกำกับของ Rules Engine ตามกติกาแบบ Lite 5e
2. เพื่อออกแบบและพัฒนา Rules Engine และ Game State Tracker ที่สามารถบังคับใช้และติดตามกลไกหลักของเกมได้อย่างถูกต้องและต่อเนื่อง เช่น การตรวจสอบค่าความสามารถ, การโจมตี, ค่าความเสียหาย, พลังชีวิต และสถานะต่างๆ ของตัวละคร
3. เพื่อพัฒนาส่วนต่อประสานกับผู้ใช้ (UI) ที่ใช้งานง่ายและตอบสนองต่อการเล่นจริง ประกอบด้วยหน้าต่างแสดงข้อมูลตัวละคร (Character Sheet), แผนที่ (Map), และบันทึกเหตุการณ์ (Log) เพื่อรองรับการเล่นบนอุปกรณ์เดียว

1.3 ประโยชน์และผลที่คาดว่าจะได้รับจากงานวิจัย

1. ได้รับต้นแบบระบบ AI Dungeon Master ที่สามารถใช้งานได้จริงสำหรับผู้เล่นกลุ่มแคชชวล ช่วยลดอุปสรรคในการเริ่มต้นเล่น Tabletop RPG
2. ได้รับแนวทางการบูรณาการเทคโนโลยีสมัยใหม่ (ASR, LLM) เข้ากับระบบเชิงตรรกะ (Rules Engine, State Tracker) สำหรับประยุกต์ใช้กับเกมที่มีโครงสร้างและกฎเกณฑ์ชัดเจน
3. ได้รับระบบที่ช่วยยกระดับประสบการณ์การเล่นในบริบทที่ไม่เป็นทางการให้สะดวกและเข้าถึงง่ายยิ่งขึ้น

4. ได้รับองค์ความรู้เชิงวิศวกรรมที่เป็นรากฐานสำคัญในการต่อยอดโครงการในอนาคต เช่น การพัฒนาให้รองรับการเล่นหลายคน (Multiplayer) ผ่านเครือข่าย หรือการพัฒนาให้รองรับภาษาไทย

1.4 ขอบเขตงานวิจัย

1.4.1 ด้านระบบเกมและกติกา

1. ระบบรองรับการเล่น Tabletop RPG โดยใช้ชุดกติกาแบบ "Lite 5e" ซึ่งเป็นกฎที่คณะผู้จัดทำได้ปรับลดความซับซ้อนลง แต่ยังคงรักษากฎหลักที่จำเป็นไว้ เช่น การตรวจสอบค่าความสามารถ (d20 check), การโจมตีและความเสียหาย, ค่าพลังชีวิต (HP), และการเคลื่อนที่แบบโซน (Zone-based Movement)
2. ระบบถูกออกแบบมาเพื่อรองรับการเล่นเนื้อเรื่องสั้นที่จบในตัว (One-shot Session) และมีระบบบันทึกสถานะเกม (Save/Load) เพียงหนึ่งช่องสำหรับเล่นต่อเนื่อง

1.4.2 ด้านการปฏิสัมพันธ์กับผู้ใช้

1. อินพุตหลักของระบบคือคำสั่งเสียงภาษาอังกฤษ ซึ่งจะถูกแปลงเป็นข้อความผ่านโมดูล ASR
2. ส่วนต่อประสานกับผู้ใช้ (UI) ถูกออกแบบมาเพื่อใช้งานบนอุปกรณ์เดียว และมีองค์ประกอบหลักดังนี้
 - 1) การตั้งค่าเริ่มต้น ผู้เล่นสามารถกำหนดจำนวนตัวละคร (1-4 ตัว) และเลือกธีมของโลก/เนื้อเรื่อง (เช่น แฟนตาซี, ไซไฟ) ก่อนเริ่มเกม
 - 2) การควบคุมตัวละคร มีกลไกให้ผู้ใช้เลือกตัวละครที่ต้องการจะสั่งการก่อนพูด เพื่อให้ระบบสามารถระบุผู้กระทำได้อย่างถูกต้อง
 - 3) หน้าต่างแสดงข้อมูล (Character Sheet) แสดงค่าสถานะ, พลังชีวิต และความสามารถหลักของตัวละคร
 - 4) แผนที่แบบโซน (Zone Map) แสดงตำแหน่งของผู้เล่นและศัตรูในลักษณะโซน (เช่น ระเบียงใกล้, กลาง, ไกล) แทนการใช้ตาราง
 - 5) บันทึกเหตุการณ์ (Action Log) แสดงผลการกระทำ, ผลการทอยลูกเต๋า และการตัดสินใจตามกติกาของ Rules Engine อย่างโปร่งใส

1.4.3 ด้านเทคนิคและการวัดผล

1. ระบบจะใช้บริการ LLM ผ่าน API เป็นหลัก โดยไม่เน้นการพัฒนากราฟิกสามมิติหรือแอนิเมชันที่ซับซ้อน แต่ให้ความสำคัญกับความสะดวกและความเข้าใจง่ายในการเล่น
2. ความสำเร็จของต้นแบบจะถูกวัดผลผ่านเกณฑ์ชี้วัดเชิงหน้าที่ (Functional Criteria) ดังนี้

- 1) ความถูกต้องของกฎ (Rule Adherence) Rules Engine ต้องสามารถตัดสินใจและคำนวณผลลัพธ์ตามกฎ Lite 5e ที่กำหนดไว้ได้อย่างถูกต้องแม่นยำ
- 2) ความสมบูรณ์ของสถานะ (State Integrity) Game State Tracker ต้องสามารถบันทึกและปรับปรุงข้อมูลสถานะของเกมได้อย่างถูกต้อง ไม่เกิดข้อผิดพลาดร้ายแรงตลอดการเล่น
- 3) การจัดการลำดับการเล่น (Flow Management): ระบบต้องสามารถนำเสนอทางเลือกและการกระทำที่ชัดเจน ทำให้ผู้เล่นสามารถดำเนินเกมไปในแต่ละรอบ (Turn) จนจบได้อย่างราบรื่นโดยไม่เกิดสถานะหยุดชะงักที่เกิดจากตัวระบบเอง

1.5 ข้อตกลงเบื้องต้น

1. ในการดำเนินโครงการนี้จะใช้ โมเดลโอเพนซอร์สสำเร็จรูป สำหรับระบบการรู้จำเสียงพูด (ASR) และใช้บริการ API (Application Programming Interface) จากผู้ให้บริการภายนอกสำหรับแบบจำลองภาษาขนาดใหญ่ (LLM) โดยจะเลือกใช้โมเดลที่มีค่าใช้จ่ายไม่สูง (เช่น Gemini Flash)
2. ผู้ใช้งานระบบจำเป็นต้องมี API Key ของตนเองสำหรับ LLM และรับผิดชอบค่าใช้จ่ายที่เกิดขึ้นจากการใช้บริการ API ดังกล่าว โดยใน UI จะมีช่องสำหรับผู้ใส่กรอก API Key
3. ต้นแบบของระบบที่พัฒนาขึ้นจะมุ่งเน้นประสบการณ์การเล่นบนอุปกรณ์เดียว (Single-device experience) โดยผู้เล่นหนึ่งคนสามารถควบคุมตัวละครได้หลายตัว หรือผู้เล่นหลายคนสามารถสลับกันเล่นได้
4. ชุดกติกา "Lite 5e" ที่ใช้ในโครงการจะเป็นชุดกติกาที่คณะผู้จัดทำได้นิยามและกำหนดขอบเขตขึ้นโดยอ้างอิงจากกลไกหลักของเกม Dungeons & Dragons ฉบับที่ 5

1.6 ขั้นตอนการดำเนินงาน

ในกระบวนการดำเนินงานวิจัยนี้ ได้ทำการแบ่งขั้นตอนการดำเนินงานเพื่อเพิ่มประสิทธิภาพของงาน โดยมีรายละเอียดดังนี้

1. ทบทวนทฤษฎีและงานวิจัยที่เกี่ยวข้องกับเทคโนโลยีที่จะนำมาใช้ในโครงการ
2. กำหนดคุณสมบัติของระบบที่ต้องการ (Requirements) และเกณฑ์วัดความสำเร็จ (Success Criteria)
3. ออกแบบสถาปัตยกรรมของระบบ (System Architecture) และแผนผังการไหลของข้อมูล (Data Flow)
4. พัฒนาด้านแบบขั้นที่หนึ่ง (Prototype-A) ซึ่งประกอบด้วยการเชื่อมต่อบนระบบ ASR, LLM และระบบบันทึกการทอยลูกเต๋า

5. พัฒนา Rules Engine และ Game State Tracker
6. พัฒนาส่วนต่อประสานกับผู้ใช้ (UI) ซึ่งประกอบด้วย Character Sheet, Zone Map, และ Battle Log
7. บูรณาการทุกส่วนประกอบเป็นต้นแบบขั้นที่สอง (Prototype-B) และทำการทดสอบการทำงานเบื้องต้น
8. ทดลองใช้งานระบบ (Pilot Test) เพื่อรวบรวมผลตอบรับและข้อเสนอแนะสำหรับนำมาปรับปรุง
9. ปรับปรุงแก้ไขระบบขั้นสุดท้าย และจัดทำรายงานโครงการฉบับสมบูรณ์

1.7 นิยามศัพท์เฉพาะ

1. AI Dungeon Master (AI DM) หมายถึง ระบบปัญญาประดิษฐ์ที่ทำหน้าที่เป็นผู้ดำเนินเกมและผู้ตัดสินใจในเกม Tabletop RPG
2. Tabletop RPG (เทเบิลท็อปอาร์พีจี) หมายถึง เกมสวมบทบาทที่เล่นบนโต๊ะ โดยอาศัยการเล่าเรื่อง การทอยลูกเต๋า และจินตนาการของผู้เล่นเป็นหลัก
3. Automatic Speech Recognition (ASR) หมายถึง เทคโนโลยีการรู้จำเสียงพูดอัตโนมัติ ทำหน้าที่แปลงสัญญาณเสียงพูดให้เป็นข้อความ
4. Large Language Model (LLM) หมายถึง แบบจำลองภาษาขนาดใหญ่ เป็นปัญญาประดิษฐ์ที่เชี่ยวชาญด้านการเข้าใจและสร้างข้อความที่เหมือนมนุษย์
5. Rules Engine (เอ็นจินกฎกติกา) หมายถึง ส่วนประกอบของซอฟต์แวร์ที่สร้างขึ้นเพื่อบังคับใช้กฎกติกาและกลไกของเกมโดยเฉพาะ
6. Game State Tracker (ระบบติดตามสถานะเกม) หมายถึง ส่วนประกอบของซอฟต์แวร์ที่ทำหน้าที่บันทึกและจัดการข้อมูลทั้งหมดที่เกิดขึ้นในเกมแบบเรียลไทม์
7. Lite 5e หมายถึง ชุดกติกาที่ถูกปรับลดความซับซ้อนลงจาก Dungeons & Dragons ฉบับที่ 5 เพื่อให้เหมาะกับผู้เล่นกลุ่มแคชวล

บทที่ 2 ทฤษฎี/งานวิจัยที่เกี่ยวข้อง

ในบทนี้จะกล่าวถึงทฤษฎีพื้นฐานและเทคโนโลยีที่เกี่ยวข้องซึ่งเป็นหัวใจสำคัญในการพัฒนาโครงการ "AI Dungeon Master" รวมถึงทบทวนงานวิจัยและโครงการที่มีอยู่ก่อนหน้านี้ เพื่อวิเคราะห์แนวทางการพัฒนา จุดแข็ง จุดอ่อน และนำมาเป็นแนวทางในการออกแบบสถาปัตยกรรมของระบบในโครงการนี้ เนื้อหาจะแบ่งออกเป็นสองส่วนหลัก ได้แก่ ส่วนของทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง และส่วนของงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

โครงการนี้เป็นการบูรณาการเทคโนโลยีปัญญาประดิษฐ์หลายแขนงเข้าด้วยกัน เพื่อสร้างระบบที่สามารถทำหน้าที่อันซับซ้อนของ Dungeon Master ได้ เทคโนโลยีหลักที่นำมาใช้ประกอบด้วย แบบจำลองภาษาขนาดใหญ่ (LLM), การรู้จำเสียงพูดอัตโนมัติ (ASR), และสถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน (Hybrid AI Architecture)

2.1.1 แบบจำลองภาษาขนาดใหญ่ (Large Language Models - LLM)

แบบจำลองภาษาขนาดใหญ่ หรือ LLM คือแบบจำลองปัญญาประดิษฐ์ประเภทหนึ่งที่ได้รับการฝึกฝนด้วยข้อมูลข้อความจำนวนมาก ทำให้มีความสามารถสูงในการทำความเข้าใจและสร้างภาษาที่ใกล้เคียงกับมนุษย์ แบบจำลองที่มีชื่อเสียงในปัจจุบัน เช่น GPT (Generative Pre-trained Transformer) ของ OpenAI หรือ Gemini ของ Google ล้วนมีพื้นฐานมาจากสถาปัตยกรรมที่เรียกว่า Transformer ซึ่งเป็นปัจจัยสำคัญที่ทำให้ LLM สามารถจัดการกับความซับซ้อนของภาษาได้ดี

1. จุดแข็งสำหรับโครงการ

1) การสร้างสรรค์เนื้อหา (Creative Text Generation) จุดแข็งที่สำคัญที่สุดของ LLM คือความสามารถในการสร้างเรื่องราว, คำบรรยายฉาก, และบทสนทนาของตัวละคร (NPC) ได้อย่างสร้างสรรค์และต่อเนื่อง ทำให้โลกของเกมมีความสมจริงและน่าสนใจ การทำงานแบบ Probabilistic ซึ่งอิงตามรูปแบบภาษาที่ได้เรียนรู้มา ช่วยให้ LLM สามารถสร้างผลลัพธ์ที่หลากหลายและคาดไม่ถึง ซึ่งเป็นหัวใจของความสนุกในเกม Tabletop RPG ที่ผู้เล่นมักจะกระทำการนอกกรอบที่คาดไว้ ตัวอย่างเช่น หากผู้เล่นตัดสินใจป็นหน้าดำของโรงเตี๊ยมแทนที่จะเดินเข้าประตูหน้า LLM สามารถ

สร้างคำบรรยายถึงสภาพของหน้าต่างที่เก่าแก่ เสียงไม้ที่ลั่นเอี๊ยด และลมเย็นที่พัดเข้ามาได้ในทันทีโดยไม่ต้องมีการเตรียมสคริปต์ไว้ล่วงหน้า

2) การจัดการบทสนทนา (Dialogue Management) LLM สามารถเข้าใจเจตนาของผู้เล่นจากคำสั่งที่เป็นภาษาธรรมชาติ และสร้างการตอบสนองที่สอดคล้องกับสถานการณ์ได้อย่างยืดหยุ่น ซึ่งแตกต่างอย่างสิ้นเชิงจากระบบบทสนทนาแบบต้นไม้ (Dialogue Tree) ในเกมดั้งเดิมที่ผู้เล่นถูกจำกัดให้เลือกตอบตามตัวเลือกที่กำหนดไว้ล่วงหน้าเท่านั้น ความสามารถนี้ทำให้ตัวละครในเกม (NPC) มีชีวิตชีวาและตอบสนองได้อย่างสมจริง ตัวอย่างเช่น แทนที่จะมีเพียงตัวเลือกสนทนา 3 ข้อ ผู้เล่นสามารถถามคำถามปลายเปิดกับ NPC เช่น "คุณเคยได้ยินข่าวลือเกี่ยวกับมังกรบนภูเขาบ้างไหม?" และ LLM ก็สามารถสร้างคำตอบที่สอดคล้องกับบทบาทและบุคลิกของ NPC ตัวนั้นได้ทันที

2. ข้อจำกัดและประเด็นที่ต้องพิจารณา

1) ภาวะไร้สถานะ (Statelessness) และข้อจำกัดของหน่วยความจำ: LLM ไม่มีหน่วยความจำถาวร มันจะจดจำข้อมูลได้เท่าที่มีอยู่ใน "หน้าต่างบริบท" (Context Window) เท่านั้น ซึ่งเป็นเหมือนหน่วยความจำระยะสั้นที่มีขนาดจำกัด เมื่อการสนทนาหรือเรื่องราวดำเนินไปนานขึ้น ข้อมูลในช่วงแรกๆ จะหลุดออกจากหน้าต่างบริบท ทำให้ LLM อาจ "ลืม" เหตุการณ์สำคัญ ชื่อตัวละคร หรือข้อมูลที่เคยเกิดขึ้นไปแล้วได้ ปัญหานี้เป็นอุปสรรคสำคัญต่อการสร้างเรื่องราวที่ต่อเนื่องและสมเหตุสมผลในระยะยาว

2) การยึดถือกฎกติกา (Rule Adherence): โดยธรรมชาติแล้ว LLM เป็นแบบจำลองเชิงความน่าจะเป็น (Probabilistic Model) ไม่ใช่ระบบเชิงตรรกะ (Logical System) ทำให้มันไม่สามารถรับประกันได้ว่าจะปฏิบัติตามกฎกติกาที่ซับซ้อนและตายตัวของเกมได้อย่างถูกต้อง 100% บ่อยครั้งที่ LLM อาจสร้างผลลัพธ์ที่ดูสมเหตุสมผลแต่ผิดหลักกลไกของเกม หรือที่เรียกว่า "Hallucination" ซึ่งเป็นสิ่งที่ไม่สามารถยอมรับได้ในบริบทของเกมที่ต้องการความยุติธรรม

ข้อจำกัดทั้งสองข้อนี้เป็นความท้าทายหลักเชิงสถาปัตยกรรม และเป็นเหตุผลสำคัญที่โครงการนี้จำเป็นต้องมีส่วนประกอบอื่นเข้ามาทำงานร่วมกับ LLM แทนที่จะใช้ LLM เพียงอย่างเดียว

2.1.2 การรู้จำเสียงพูดอัตโนมัติ (Automatic Speech Recognition - ASR)

ASR คือเทคโนโลยีที่ทำหน้าที่แปลงสัญญาณเสียงพูดของมนุษย์ให้กลายเป็นข้อมูลข้อความดิจิทัล เพื่อให้คอมพิวเตอร์สามารถนำไปประมวลผลต่อได้ ในบริบทของโครงการนี้ ASR จะทำหน้าที่เป็น "หู" ของ AI DM ทำให้ผู้เล่นสามารถสั่งการตัวละครด้วยเสียงพูดได้โดยตรง สร้างประสบการณ์ที่ลื่นไหลและเป็นธรรมชาติกว่าการพิมพ์

ในปัจจุบัน เทคโนโลยี ASR สามารถแบ่งได้เป็น 2 ประเภทหลัก คือแบบที่อาศัยบริการคลาวด์ (Cloud-based API) และแบบโมเดลโอเพนซอร์สที่ประมวลผลบนอุปกรณ์ของผู้ใช้โดยตรง (On-device) สำหรับโครงการนี้ เราได้เลือกใช้ โมเดลโอเพนซอร์ส เพื่อให้ระบบสามารถทำงานได้ด้วยตนเองโดยไม่ต้องพึ่งพาอินเทอร์เน็ตตลอดเวลา และเพื่อความเป็นส่วนตัวของผู้ใช้งาน การเลือกระหว่างโมเดลยอดนิยม เช่น Whisper ของ OpenAI ซึ่งมีความแม่นยำสูงแต่ต้องการทรัพยากรเครื่องที่มาก กับ Vosk ซึ่งมีขนาดเล็กและทำงานได้รวดเร็วบน CPU ทั่วไป แต่ต้องมีการปรับจูนเพิ่มเติมเพื่อความแม่นยำในศัพท์เฉพาะทาง ถือเป็นการตัดสินใจเชิงสถาปัตยกรรมที่ต้องพิจารณาถึงความสมดุลระหว่างประสิทธิภาพและความเข้าถึงง่ายของผู้ใช้

2.1.3 สถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน (Hybrid AI Architecture)

เพื่อแก้ไขข้อจำกัดของ LLM ที่กล่าวไปข้างต้น โครงการนี้จึงได้นำแนวคิดของ สถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน หรือที่รู้จักในอีกชื่อหนึ่งว่า Neuro-Symbolic AI มาใช้ แนวคิดนี้คือการนำระบบสองประเภทที่มีความถนัดต่างกันมาทำงานร่วมกัน เปรียบได้กับการทำงานของสมองมนุษย์สองส่วน (System 1 และ System 2 Thinking)

1. Neural Component (LLM) เปรียบเสมือน System 1 คือส่วนของโครงข่ายประสาทเทียมที่ทำงานอย่างรวดเร็ว เป็นธรรมชาติ และใช้สัญชาตญาณ ในที่นี้คือ LLM ซึ่งทำหน้าที่เกี่ยวกับความคิดสร้างสรรค์ การเข้าใจภาษา และการสร้างเนื้อหาเชิงพรรณนา
2. Symbolic Component (Code) เปรียบเสมือน System 2 คือส่วนที่ทำงานอย่างช้าๆ เป็นเหตุเป็นผล และอยู่บนพื้นฐานของตรรกะและกฎเกณฑ์ที่กำหนดไว้อย่างชัดเจน ในโครงการนี้หมายถึง Rules Engine และ Game State Tracker ที่เขียนขึ้นด้วยโค้ดโปรแกรม

สถาปัตยกรรมแบบผสมผสานนี้จะแบ่งหน้าที่ความรับผิดชอบอย่างชัดเจน

1. LLM รับผิดชอบในส่วนของ "การเล่าเรื่อง" (Narrative) เช่น การบรรยายฉาก, การตอบโต้ของ NPC
2. Rules Engine & Game State Tracker รับผิดชอบในส่วนของ "กลไกเกมและหน่วยความจำ" (Mechanics & Memory) เช่น การคำนวณความเสียหาย, การตรวจสอบผลการทอยลูกเต๋า, และการบันทึกว่า "ตอนนี้ผู้เล่นมีพลังชีวิตเหลือเท่าไร"

ด้วยสถาปัตยกรรมนี้ ระบบจะให้ LLM ทำในสิ่งที่ถนัด และใช้โค้ดโปรแกรมที่ทำงานอย่างแม่นยำมาจัดการในส่วนที่ LLM ทำได้ไม่ดี ซึ่งเป็นแนวทางที่ได้รับการยอมรับอย่างกว้างขวางในการพัฒนาระบบ AI ที่ต้องการทั้งความคิดสร้างสรรค์และความน่าเชื่อถือ

2.1.4 การสร้างข้อความด้วยการดึงข้อมูลมาเสริม (Retrieval-Augmented Generation - RAG)

RAG เป็นเทคนิคที่กำลังได้รับความนิยมอย่างสูงในการเพิ่มประสิทธิภาพและความน่าเชื่อถือให้กับ LLM หลักการของ RAG คือ แทนที่จะปล่อยให้ LLM ตอบคำถามจาก "ความรู้" ที่มีอยู่ภายในเพียงอย่างเดียว ระบบจะทำการ ดึงข้อมูลที่เกี่ยวข้อง จากแหล่งข้อมูลภายนอก (เช่น เอกสาร, ฐานข้อมูล) แล้วนำข้อมูลนั้นมาประกอบในคำสั่ง (Prompt) เพื่อให้ LLM ใช้เป็นบริบทในการสร้างคำตอบ ในโครงงานนี้ RAG จะเป็นกลไกสำคัญที่เชื่อมต่อระหว่าง LLM และ Symbolic Component

1. การดึงกฎกติกา (Rule Retrieval): เมื่อผู้เล่นต้องการกระทำการบางอย่างที่เกี่ยวข้องกับกฎ เช่น "ร่ายเวทมนตร์ Fireball" ระบบ RAG สามารถดึงข้อมูลกฎของเวทมนตร์ Fireball จากฐานข้อมูล แล้วส่งไปพร้อมกับคำสั่งให้ LLM เพื่อให้มันบรรยายผลลัพธ์ได้ถูกต้องตามกฎ เช่น Damage: 8d6 fire damage, Range: 150 feet
2. การดึงสถานะของเกม (State Retrieval): ในทุกๆ การกระทำของผู้เล่น ระบบ RAG จะดึงข้อมูลสถานะล่าสุดของเกมจาก Game State Tracker (เช่น Player HP: 12/15, Goblin HP: 5/7) มาประกอบในคำสั่ง เพื่อให้ LLM "รับรู้" สถานการณ์ปัจจุบันและสร้างเนื้อเรื่องที่สอดคล้องกับสิ่งที่เกิดขึ้นจริงในเกม เช่น การบรรยายว่าก๊อบลิน "ดูบาดเจ็บสาหัส" เมื่อพลังชีวิตเหลือน้อย

การใช้เทคนิค RAG ทำให้เราสามารถแก้ปัญหา Long-term Memory ของ LLM ได้อย่างมีประสิทธิภาพ และยังเป็นกลไกสำคัญในการกำกับให้ LLM สร้างเนื้อหาที่สอดคล้องกับกฎและสถานะของเกมอยู่เสมอ

2.2 งานวิจัยและโครงงานที่เกี่ยวข้อง

ในช่วงไม่กี่ปีที่ผ่านมา มีความพยายามในการนำ LLM มาประยุกต์ใช้เป็น Dungeon Master หลายโครงการ การทบทวนงานวิจัยเหล่านี้ช่วยให้เห็นถึงแนวทาง ความท้าทาย และบทเรียนที่สามารถนำมาปรับใช้กับโครงงานนี้ได้

2.2.1 Triyason (2023) - การใช้ ChatGPT เป็น DM สำหรับผู้เล่นคนเดียว

งานวิจัยชิ้นแรกๆ ที่ทดลองใช้ ChatGPT ในการดำเนินเกม D&D พบว่า LLM สามารถสร้างเรื่องราวที่ต่อเนื่องและตอบสนองต่อการกระทำของผู้เล่นได้ดีพอที่จะทำให้ผู้เล่นใหม่ได้สัมผัสกับประสบการณ์ D&D อย่างไรก็ตาม ข้อจำกัดที่พบคือการตอบสนองที่ล่าช้าในบางครั้ง และการขาดการแสดงอารมณ์

ร่วม (Limited Emotional Engagement) ซึ่งส่งผลต่อความอินของผู้เล่น งานวิจัยนี้ชี้ให้เห็นถึงศักยภาพของ LLM แต่ก็ยังถึงความจำเป็นในการปรับปรุงด้านความเร็วและความสามารถในการเล่าเรื่องให้ น่าดึงดูดยิ่งขึ้น

2.2.2 Sakellaridis (2024) - การเปรียบเทียบ LLM ที่ผ่านการ Fine-tune กับ DM มนุษย์

งานวิจัยนี้ได้ทำการฝึกฝน (Fine-tune) LLM ด้วยข้อมูลบทสนทนาจากเกม D&D และข้อมูลกฎกติกา โดยเฉพาะ แล้วนำมาเปรียบเทียบกับการเล่นกับ DM ที่เป็นมนุษย์ ผลลัพธ์ที่น่าสนใจคือ ผู้เล่นให้คะแนนความคืบหน้ากับคำบรรยายจากของ AI DM สูงเทียบเท่าหรือสูงกว่า DM มนุษย์เสียอีก แสดงให้เห็นว่า LLM มีความเป็นเลิศในการสร้างสรรค์คำบรรยาย แต่ข้อเสียที่พบคือ AI มักจะดำเนินเรื่องไปในทิศทางที่กำหนดไว้ล่วงหน้า (Railroading) ไม่ยืดหยุ่นต่อการตัดสินใจนอกกรอบของผู้เล่น และสร้างความท้าทายในเกมได้ไม่ดีพอ

2.2.3 Jorgensen et al. (2024) - สถาปัตยกรรมแบบ Multi-Agent

อีกหนึ่งแนวทางที่น่าสนใจคือการออกแบบ AI DM ด้วยสถาปัตยกรรมแบบหลายเอเจนต์ (Multi-Agent) โดยใช้เฟรมเวิร์กอย่าง ReAct เพื่อให้ AI สามารถ "คิด" และ "กระทำ" เป็นขั้นเป็นตอนได้ โดยอาจมีเอเจนต์ที่ทำหน้าที่เฉพาะทาง เช่น เอเจนต์สำหรับเล่าเรื่อง, เอเจนต์สำหรับจัดการกฎกติกา ซึ่งช่วยให้ระบบโดยรวมทำงานได้อย่างสอดคล้องและน่าเชื่อถือมากขึ้น แนวทางนี้สนับสนุนการออกแบบเชิงโมดูลของโครงการเรา

2.2.4 Tool-Assisted AI DM (2024) - การใช้ RAG และ Function Calling

งานวิจัยล่าสุดได้นำเสนอแนวคิด AI DM ที่สามารถเรียกใช้เครื่องมือภายนอก (External Tools) ได้ เช่น การเรียกฟังก์ชันสำหรับทอยลูกเต๋า (Function Calling) หรือการใช้ RAG เพื่อดึงกฎกติกาจากฐานข้อมูลตามเวลาจริง ผลการทดลองพบว่าสถาปัตยกรรมแบบผสมผสานนี้ช่วยให้ AI DM มีความสม่ำเสมอในการรักษาสถานะของเกม และปฏิบัติตามกฎได้อย่างน่าเชื่อถือสูงกว่าการใช้ LLM เพียงอย่างเดียว

2.2.5 สรุปและบทเรียนที่ได้รับจากงานวิจัยที่เกี่ยวข้อง

จากงานวิจัยทั้งหมดที่กล่าวมา สามารถสรุปบทเรียนสำคัญได้ว่า การใช้ LLM เพียงอย่างเดียวไม่น่าเพียงพอ ที่จะสร้างประสบการณ์การเล่น Tabletop RPG ที่สมบูรณ์ได้ แม้ LLM จะมีความสามารถในการเล่าเรื่องที่ยอดเยี่ยม แต่ก็มีจุดอ่อนที่ชัดเจนในด้านการรักษาสถานะ, การปฏิบัติตามกฎ, และความยืดหยุ่นในการตอบสนองต่อการกระทำของผู้เล่น การพยายามใช้ LLM เพียงอย่างเดียวมักจะทำให้ผลลัพธ์เป็นเพียง "ผู้ช่วยเล่าเรื่อง" แต่ไม่ใช่ "ผู้ดำเนินเกม" ที่แท้จริง

แนวทางที่ประสบความสำเร็จและได้รับการยอมรับมากที่สุดคือ สถาปัตยกรรมแบบผสมผสาน (Hybrid Architecture) ที่ใช้ LLM ทำหน้าที่สร้างสรรค์เนื้อหา และใช้ระบบเชิงตรรกะที่เขียนด้วยโค้ด (Symbolic System) มาจัดการกับกลไกและสถานะของเกม โดยมีเทคนิคอย่าง RAG และ Function Calling เป็นสะพานเชื่อมระหว่างสองส่วนนี้ ข้อมูลเชิงลึกเหล่านี้ได้กลายมาเป็นรากฐานสำคัญในการออกแบบสถาปัตยกรรมของโครงการนี้ เพื่อสร้างระบบ AI Dungeon Master ที่มีความสามารถรอบด้านและเอาชนะข้อจำกัดที่พบในงานวิจัยก่อนหน้านี้

ลำดับ	แผนดำเนินงาน	2568				2569				
		ก.ย.	ต.ค.	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย	พ.ค.
6.	บูรณาการระบบเป็น Prototype-B และ ทดสอบเบื้องต้น									
7.	ทดลองใช้งานระบบ (Pilot Test) และ รวบรวมผลตอบรับ									
8.	ปรับปรุงระบบขั้นสุดท้าย และจัดทำ รายงานฉบับสมบูรณ์									

รูปที่ X แผนภูมิแกนต์

3.2 เครื่องมือและเทคโนโลยีที่ใช้

การพัฒนาโครงงานนี้อาศัยเครื่องมือซอฟต์แวร์และเทคโนโลยีที่หลากหลาย ซึ่งถูกคัดเลือกมาโดยคำนึงถึงเป้าหมายหลักคือการสร้างแอปพลิเคชันที่ทำงานได้ด้วยตนเองบนหลายแพลตฟอร์ม (Cross-platform) และง่ายต่อการนำไปใช้งาน

1. ภาษาโปรแกรมและเฟรมเวิร์ก Dart เป็นภาษาหลักในการพัฒนา โดยใช้เฟรมเวิร์ก Flutter ในการสร้างแอปพลิเคชันส่วนหน้า (Frontend) เหตุผลที่เลือกใช้ Flutter เนื่องจากมีจุดแข็งในการใช้ฐานโค้ดเดียว (Single Codebase) แต่สามารถคอมไพล์เป็นแอปพลิเคชัน ที่ทำงานได้บนหลายระบบปฏิบัติการ เช่น Windows, macOS, Android และ iOS ซึ่งตอบโจทย์เป้าหมายของโครงงานที่ต้องการให้ผู้ใช้สามารถเข้าถึงได้ง่าย
2. ส่วนประมวลผลหลัก (Backend Logic) โลจิกหลักของเกม เช่น Rules Engine และ State Tracker จะถูกพัฒนาด้วยภาษา Dart เช่นกัน เพื่อให้เป็นส่วนหนึ่งของแอปพลิเคชัน Flutter โดยตรง แนวทางนี้ช่วยลดความซับซ้อนของสถาปัตยกรรมลงอย่างมาก โดยไม่จำเป็นต้องมีการตั้งเซิร์ฟเวอร์แยกต่างหาก และทำให้แอปพลิเคชันสามารถทำงานได้อย่างสมบูรณ์ในตัวเอง (Self-contained)
3. การรู้จำเสียงพูด (ASR) ใช้โมเดลโอเพนซอร์สสำเร็จรูปที่สามารถทำงานบนอุปกรณ์ได้โดยตรง (On-device) โดยจะทำการศึกษาและคัดเลือกไลบรารีที่มีประสิทธิภาพและสามารถผนวกรวมเข้ากับ Flutter ได้ เพื่อให้ผู้ใช้ไม่ต้องเชื่อมต่ออินเทอร์เน็ตสำหรับฟังก์ชันนี้และเพื่อรักษาความเป็นส่วนตัวของข้อมูลเสียง

4. แบบจำลองภาษาขนาดใหญ่ (LLM) เรียกใช้งานผ่าน REST API ไปยังผู้ให้บริการภายนอก โดยจะเลือกใช้โมเดลที่มีค่าใช้จ่ายไม่สูง เช่น Gemini Flash การเชื่อมต่อจะพัฒนาบน Flutter โดยใช้แพ็คเกจสำเร็จรูป เช่น flutter_gemini เพื่อจัดการการสื่อสารกับ API ซึ่งช่วยลดความซับซ้อนในการพัฒนาและทำให้สามารถมุ่งเน้นไปที่การสร้างตรรกะของเกมได้โดยตรง
5. การแปลงข้อความเป็นเสียง (Text-to-Speech - TTS): จะมีการเพิ่มฟังก์ชันเสริมในการอ่านออกเสียงข้อความบรรยายจาก AI DM ด้วยเสียงสังเคราะห์แบบพื้นฐาน โดยอาศัยไลบรารีมาตรฐานที่มีใน Flutter เพื่อเพิ่มทางเลือกในการรับรู้ข้อมูลของผู้เล่นโดยไม่เพิ่มความซับซ้อนให้กับระบบมากนัก

3.3 การออกแบบและพัฒนาระบบ

หัวใจของระเบียบวิธีวิจัยนี้คือการออกแบบสถาปัตยกรรมซอฟต์แวร์ที่สามารถบูรณาการเทคโนโลยีต่างๆ เข้าด้วยกันได้อย่างมีประสิทธิภาพ

3.3.1 ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview)

ระบบ AI Dungeon Master ถูกออกแบบตามสถาปัตยกรรมแบบผสมผสาน (Hybrid Architecture) ซึ่งเป็นแนวทางที่ได้รับการยอมรับอย่างกว้างขวางในการพัฒนาระบบ AI ที่ซับซ้อน โดยมีหลักการสำคัญคือการแยกส่วนของการเล่าเรื่อง (Narrative Engine) ซึ่งอาศัยความสามารถเชิงสร้างสรรค์ของ LLM ออกจากส่วนของกลไกเกม (Game Logic Engine) ซึ่งต้องการความแม่นยำและตรรกะที่ตายตัว การแบ่งหน้าที่ที่ชัดเจนนี้ช่วยให้แต่ละส่วนประกอบทำงานที่ตนเองถนัดได้อย่างเต็มประสิทธิภาพและลดข้อผิดพลาดที่อาจเกิดขึ้นจากธรรมชาติของเทคโนโลยี การไหลของข้อมูล (Data Flow) ในระบบจะเป็นไปตามลำดับขั้นตอนดังแสดงในแผนภาพ และคำอธิบายต่อไปนี้

1. รับอินพุตจากผู้เล่น (Player Input) กระบวนการเริ่มต้นเมื่อผู้เล่นกดปุ่มเลือกตัวละครที่ต้องการสังการ จากนั้นส่งคำสั่งเสียงเข้ามาในระบบผ่านไมโครโฟนของอุปกรณ์
2. การแปลงเสียงเป็นข้อความ (ASR Processing) โมดูล ASR ซึ่งทำงานบนอุปกรณ์ของผู้ใช้โดยตรง จะทำการแปลงสัญญาณเสียงพูดให้เป็นข้อความดิบ (Raw Text) เช่น "I want to attack the goblin with my sword."
3. การประมวลผลคำสั่ง (Command Parsing) ระบบทำการวิเคราะห์ข้อความดิบเพื่อสกัดเจตนาหลัก (Intent) และองค์ประกอบที่เกี่ยวข้อง (Entities) ของผู้เล่นออกมาเป็นโครงสร้างที่ชัดเจน เช่น { "action": "attack", "target": "goblin", "weapon": "sword" } ข้อมูลที่มีโครงสร้างนี้จะถูกส่งไปให้ Rules Engine เพื่อใช้ในการคำนวณเชิงกลไกต่อไป

4. การดึงข้อมูลเพื่อสร้างบริบท (Context Retrieval - RAG): ขั้นตอนนี้คือหัวใจสำคัญของสถาปัตยกรรมแบบผสมผสาน ก่อนที่จะส่งคำสั่งไปยัง LLM ระบบจะทำการ "Augment" หรือเสริมบริบทเข้าไปในคำสั่งโดยอัตโนมัติ โดยดึงข้อมูลที่เกี่ยวข้องจาก 3 แหล่งหลัก:

1) สถานะปัจจุบันของเกม (Dynamic Context) ดึงข้อมูลล่าสุดจาก Game State Tracker เช่น Player HP: 12/15, Goblin HP: 5/7, Player Location: Cave Entrance การทำเช่นนี้เป็นการแก้ปัญหาภาวะไร้สถานะ (Statelessness) ของ LLM ทำให้มัน "รับรู้" สถานการณ์ล่าสุดของเกมได้เสมอ

2) ประวัติการสนทนา (Historical Context) ดึงข้อมูลบทสนทนาและการกระทำล่าสุด 2-3 รอบย้อนหลัง เพื่อให้ LLM สามารถสร้างเนื้อเรื่องที่ต่อเนื่องและสอดคล้องกับสิ่งที่เพิ่งเกิดขึ้น

3) กฎกติกาที่เกี่ยวข้อง (Static Context) ในกรณีที่การกระทำของผู้เล่นเกี่ยวข้องกับกฎเฉพาะทาง ระบบ RAG จะทำการค้นหาและดึงเฉพาะส่วนของกฎนั้นๆ จากฐานข้อมูลกติกา "Lite 5e" เช่น หากผู้เล่นร่ายเวท "Magic Missile" ระบบจะดึงคำอธิบายของเวทมนตร์นี้มาประกอบใน Prompt เพื่อให้ LLM บรรยายผลลัพธ์ได้ถูกต้องตามกฎ

5. การสร้างคำสั่งไปยัง LLM (Prompt Generation) ระบบจะนำข้อมูลทั้งหมดที่ได้จากขั้นตอนก่อนหน้านี้ (ข้อความคำสั่งจากผู้เล่น, สถานะเกม, ประวัติการสนทนา, และกฎที่เกี่ยวข้อง) มาประกอบกันเป็น Prompt ที่สมบูรณ์ตามแม่แบบ (Template) ที่ออกแบบไว้เพื่อส่งไปยัง LLM API

6. การสร้างเนื้อหาโดย LLM (Narrative Generation): LLM รับ Prompt ที่มีบริบทครบถ้วน และสร้างคำบรรยายหรือบทสนทนาที่สอดคล้องกับสถานการณ์นั้นๆ เช่น "You swing your rusty sword towards the goblin..."

7. การบังคับใช้กฎและอัปเดตสถานะ (Rule Enforcement & State Update) ขั้นตอนนี้คือการทำงานของระบบเชิงตรรกะ (Symbolic System) ที่เกิดขึ้น ควบคู่ไปกับ การสร้างเนื้อหาโดย LLM ส่วนของ Rules Engine จะรับข้อมูลคำสั่งที่มีโครงสร้างจากขั้นตอนที่ 3 มาประมวลผลเชิงกลไกอย่างแม่นยำ เช่น ทำการสุ่มตัวเลขสุ่ม (RNG) เพื่อจำลองการทอยลูกเต๋า d20, เปรียบเทียบผลลัพธ์กับค่า Armor Class ของก๊อบลินที่เก็บอยู่ใน Game State Tracker, คำนวณความเสียหายที่เกิดขึ้น, และสุดท้ายคือการอัปเดตค่าพลังชีวิตใหม่ลงใน Game State Tracker โดยตรง ขั้นตอนนี้รับประกันได้ว่ากลไกของเกมจะทำงานอย่างถูกต้องและยุติธรรมเสมอ แยกขาดจากการสร้างเนื้อหาเชิงพรรณนาของ LLM

8. การแสดงผลลัพธ์ (Output to UI) สุดท้าย ระบบจะนำผลลัพธ์จากทั้งสองส่วนมารวมกันเพื่อแสดงผลบน UI ได้แก่ คำบรรยายจาก LLM, ผลการทอยลูกเต๋าจาก Rules Engine, และข้อมูลสถานะที่อัปเดตแล้วจาก Game State Tracker นอกจากนี้ อาจมีการแปลงข้อความบรรยายเป็นเสียงพูดผ่านระบบ TTS เพื่อเพิ่มความสมจริงในการเล่น

3.3.2 การพัฒนาส่วนประกอบหลัก (Development of Key Modules)

1. Rules Engine & Game State Tracker พัฒนาเป็นส่วนหนึ่งของแอปพลิเคชัน Flutter ด้วยภาษา Dart โดย Game State Tracker จะถูกออกแบบเป็นคลาสหลัก (e.g., GameState) ที่เก็บข้อมูลทั้งหมดในรูปแบบของ Class Object ที่มีโครงสร้างชัดเจน เช่น PlayerCharacter, Enemy, Item ซึ่งทำให้การจัดการข้อมูลเป็นระบบและง่ายต่อการแก้ไข ส่วน Rules Engine จะถูกสร้างเป็นกลุ่มของฟังก์ชันเชิงตรรกะ (e.g., calculateAttackOutcome()) ที่รับข้อมูลสถานะปัจจุบันเป็นอินพุตและส่งคืนผลลัพธ์โดยไม่มีการเปลี่ยนแปลงสถานะโดยตรง (Pure Functions) เพื่อให้ง่ายต่อการทดสอบและรับประกันความถูกต้อง
2. LLM Integration and Prompt Engineering ส่วนนี้จะเน้นที่การออกแบบ "Prompt Template" ที่มีประสิทธิภาพ โดยใช้หลักการที่ศึกษามาในการแบ่งโครงสร้าง Prompt ด้วย Tag ที่ชัดเจน (เช่น <context>, <rules>, <history>, <user_action>) เพื่อชี้แนะให้ LLM สามารถแยกแยะและเข้าใจบริบทต่างๆ ได้ดีที่สุด ซึ่งจะนำไปสู่การสร้างผลลัพธ์ที่สอดคล้องกับกฎและสถานะของเกม ลดโอกาสการเกิด Hallucination
3. User Interface (UI) Development พัฒนาด้วยระบบ Widget ของ Flutter โดยเน้นการออกแบบที่สะอาดและไม่ซับซ้อน (Minimalist Design) โดยให้ความสำคัญกับลำดับชั้นของข้อมูล (Information Hierarchy) เช่น การแสดงผลค่าพลังชีวิตและค่าสถานะหลักให้เห็นได้ชัดเจนตลอดเวลา ในขณะที่ส่วนของบันทึกเหตุการณ์ (Log) สามารถเลื่อนดูย้อนหลังได้

3.4 แหล่งข้อมูลและความรู้ของระบบ (System Knowledge and Data Sources)

1. ชุดกติกา Lite 5e คณะผู้จัดทำได้ออกแบบและสร้างเอกสารสรุปกฎ Lite 5e ขึ้นมาโดยเฉพาะ ซึ่งจะทำหน้าที่เป็นเอกสารข้อกำหนด (Specification Document) สำหรับการพัฒนากลไกทั้งหมดใน Rules Engine
2. เนื้อเรื่องและสถานการณ์ (Scenario Content): สำหรับต้นแบบ (Prototype) ระบบจะถูกออกแบบให้สามารถดำเนินเรื่องราวไปได้อย่างต่อเนื่องในลักษณะ Mission-based คือเมื่อจบภารกิจหนึ่ง ผู้เล่นจะสามารถรับภารกิจใหม่เพื่อเล่นต่อไปได้เรื่อยๆ โดยไม่มีจุดจบที่ตายตัว เพื่อรองรับการเล่นแบบปลายเปิด (Open-ended) และเพื่อใช้เป็นสถานการณ์ทดสอบหลักสำหรับ Pilot Test

3.5 การทดสอบและประเมินผล (System Testing and Evaluation)

การทดสอบระบบเป็นขั้นตอนสำคัญเพื่อรับประกันคุณภาพของต้นแบบ โดยจะแบ่งออกเป็น 2 ระยะหลัก

1. การทดสอบเชิงหน้าที่ (Functional Testing): เป็นการทดสอบเพื่อรับประกันความถูกต้องของโมดูลหลัก โดยเฉพาะ Rules Engine จะใช้วิธีการเขียน Unit Test ซึ่งเป็นการสร้างชุดคำสั่งเพื่อทดสอบ

การทำงานของฟังก์ชันต่างๆ ในสถานการณ์จำลองที่กำหนดไว้ล่วงหน้า (Scripted Scenarios) แนวทางนี้มีความสำคัญอย่างยิ่งในการรับประกันว่าตรรกะหลักของเกมทำงานได้ถูกต้อง 100% ก่อนที่จะนำไปบูรณาการกับ LLM ที่มีความไม่แน่นอนสูง

2. การทดลองใช้งานระบบ (Pilot Test): หลังจากที่บูรณาการทุกส่วนเป็น Prototype-B แล้ว จะมีการจัดทดลองใช้งานระบบกับกลุ่มเป้าหมาย (ผู้เล่นแคชชวล 2-3 คน) โดยกระบวนการจะแบ่งเป็น 3 ช่วง คือ 1) การแนะนำวิธีเล่น (Onboarding), 2) การทดลองเล่นสถานการณ์ที่เตรียมไว้, และ 3) การเก็บรวมผลตอบรับ (Feedback Collection): หลังจากเล่นจบ จะให้ผู้ทดลองทำ แบบสอบถามความพึงพอใจ ซึ่งออกแบบมาเพื่อวัดผลเกณฑ์ชี้วัดความสำเร็จที่กำหนดไว้ในบทที่ 1 โดยเฉพาะ เช่น การให้คะแนนความยุติธรรมของกฎ (Rule Adherence), ความต่อเนื่องของเนื้อเรื่อง (Narrative Quality), และความง่ายในการใช้งาน (Usability) โดยใช้มาตรวัดแบบ Likert Scale (เช่น 1-5 คะแนน) ควบคู่ไปกับการสัมภาษณ์สั้นๆ เพื่อเก็บข้อมูลเชิงลึกเพิ่มเติม ผลตอบรับที่ได้จะถูกนำมาใช้วิเคราะห์เพื่อสรุปผลความสำเร็จของโครงการและเป็นแนวทางในการปรับปรุงแก้ไขระบบในขั้นตอนสุดท้าย

เอกสารอ้างอิง

บทความในรายงานการประชุมทางวิชาการและวารสาร

1. Triyason, T., et al., 2023, "**Exploring the Potential of ChatGPT as a Dungeon Master in Dungeons & Dragons Tabletop Game**", Proceedings of the 2023 International Conference on Digital Image and Multimedia (ICODIM), 1-3 December, Chiang Mai, Thailand, pp. 1-6.
2. Jørgensen, N. H., Tharmabalan, T., Aslan, M., & Merritt, T., 2024, "**Evaluating Multi-Agent Architectures for an AI Game Master in Dungeons & Dragons**", Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems, 11-16 May, Honolulu, HI, USA.
3. Lanchantin, J., et al., 2024, "**You Have Thirteen Hours in Which to Solve the Labyrinth: Enhancing AI Game Masters with Function Calling**", arXiv preprint arXiv:2409.06949.
4. Vaswani, A., et al., 2017, "**Attention Is All You Need**", Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, pp. 5998-6008.
5. Radford, A., et al., 2022, "**Robust Speech Recognition via Large-Scale Weak Supervision**", Proceedings of the 39th International Conference on Machine Learning (ICML), 17-23 July, Baltimore, Maryland, USA, pp. 18076-18093.
6. Sakellaridis, P., 2024, **AI Dungeon Master for DnD Railroads Players**, [Online], Available: <https://www.wargamer.com/dnd/ai-railroads-players> [2025, October 19].
7. Maassen, H., 2019, **D&D 5e Lite**, [Online], Available: <https://haraldmaassen.com/blog/2019/06/02/dd-5e-lite/> [2025, October 19].
8. Babakcode, 2024, **flutter_gemini 3.0.0**, [Online], Available: https://pub.dev/packages/flutter_gemini [2025, October 19].

ภาคผนวก ก

ชุดกติกาย่อ Lite 5e (ฉบับสำหรับทีลีส)

บทนำ

ยินดีต้อนรับนักผจญภัย! 🎲

คู่มือกติกาที่ถูกออกแบบมาสำหรับ ผู้เล่นใหม่ และ เพื่อวัตถุประสงค์ของโครงการนี้โดยเฉพาะ ระบบของเราได้รับแรงบันดาลใจจาก **Dungeons & Dragons ฉบับที่ 5** แต่ได้ถูก ปรับให้เรียบง่ายขึ้น เพื่อให้:

- ง่ายต่อการเรียนรู้
- เล่นได้อย่างรวดเร็ว
- เหมาะสมกับการโต้ตอบผ่าน คำสั่งเสียง กับ AI Dungeon Master (LLM)

ไม่ว่าคุณจะเป็นนักรบที่เหวี่ยงดาบ 🗡️, นักบวชผู้ร้องขอพลังจากทวยเทพ 🏰, หรือจอมเวทผู้ควบคุมความเป็นจริง 🌟 กฎเหล่านี้จะมอบโครงสร้างที่ยุติธรรมให้กับการเล่นของคุณ ในขณะที่ยังคงรักษาอิสระในการดำเนินเรื่องราวไว้

วิธีการเล่น (How to Play)

หัวใจหลักของเกมนี้เรียบง่าย:

1. LLM/DM บรรยายสถานการณ์

“ก๊อบลินตัวหนึ่งกระโดดออกมาจากหลังถ้ำ!”

2. คุณบอกสิ่งที่คุณต้องการจะทำ

“ฉันจะใช้ดาบฟันไปที่ก๊อบลิน!”

3. ระบบทอยลูกเต๋าเพื่อดูว่าคุณทำสำเร็จหรือไม่

$d20 + \text{ค่าโบนัสกายภาพ (PHYS)}$ เทียบกับ ค่าความป้องกัน (AC) ของศัตรู

4. ผลลัพธ์จะถูกบรรยายออกมา

“ดาบของคุณฟันเข้าอย่างจัง! ก๊อบลินได้รับความเสียหาย 7 หน่วย”

พื้นฐานเรื่องลูกเต๋า (Dice Basics)

เราจะใช้ระบบ **d20** เป็นหลัก:

- การตรวจสอบทักษะ (Skill Checks) → $d20 + \text{ค่าโบนัสค่าสถานะ}$ เทียบกับ ค่าความยาก (DC)
- การโจมตี (Attack Rolls) → $d20 + \text{ค่าโบนัสค่าสถานะ}$ เทียบกับ ค่าความป้องกัน (AC) ของศัตรู
- การคำนวณความเสียหาย (Damage Rolls) → ทอยลูกเต๋าสีเล็กกว่า (d6, d8)



การทอยพิเศษ:

- **ได้แต้ม 20 (Natural 20)** → สำเร็จโดยอัตโนมัติ (Critical Success) และสร้างความเสียหายสองเท่า
- **ได้แต้ม 1 (Natural 1)** → ล้มเหลวโดยอัตโนมัติ (Critical Failure) และอาจมีผลเสียตามมา

ค่าสถานะและค่าโบนัส (Stats & Modifiers)

เราได้รวมค่าความสามารถทั้ง 6 ของ D&D แบบดั้งเดิมให้เหลือเพียง **3 ค่าสถานะหลัก**:

ค่าสถานะ	ครอบคลุม	ตัวอย่างการกระทำ
กายภาพ (PHYS)	พลัง, ความคล่องแคล่ว	โจมตี, ปีนป่าย, หลบหลีก, ผลัก
สติปัญญา (MENT)	ความฉลาด, สติปัญญา	ค้นหาความรู้, สังเกตกับดัก, แก้ปริศนา
สังคม (SOC)	เสน่ห์	โน้มน้าวใจ, โกหก, ข่มขู่

- แต่ละค่าสถานะจะมี **ค่าโบนัส (Modifier)** ตั้งแต่ -2 ถึง +5
- นำค่านี้ไปบวกกับการทอยลูกเต๋าของคุณ

พลังชีวิต (Hit Points - HP)

- ตัวละครทั้งหมดเริ่มต้นที่ **20 HP** ที่เลเวล 1
- ความเสียหายจะลดค่า HP
- **HP เหลือ 0 = หหมดสติ (Unconscious)**
- หากไม่ได้รับการรักษา อาจนำไปสู่ความตายได้

ระบบการต่อสู้ (Combat System)

การต่อสู้เป็นแบบ **ผลัดกันเล่น (Turn-based)** และใช้ระบบ **โซน (Zone-based)**

ลำดับการเล่น (Turn Order)

- ผู้เล่น → ศัตรู → ผู้เล่น → ศัตรู ...
- ดำเนินไปเรื่อยๆ จนกว่าฝ่ายใดฝ่ายหนึ่งจะพ่ายแพ้หรือล่าถอย

การกระทำในแต่ละรอบ (Actions per Turn)

ในรอบของคุณ คุณสามารถทำ **1 การกระทำ (Action)** + **1 การเคลื่อนที่ (Move)**:

- โจมตี (ระยะประชิด/ระยะไกล)
- ร่ายเวท (ถ้ามี)
- ใช้ความสามารถพิเศษ
- ใช้ไอเทม (ดื่มยา, โยนขวด)
- มีปฏิสัมพันธ์กับสิ่งแวดล้อม (ถีบประตู, ดึงคันโยก)

การเคลื่อนที่ (Movement)

เราไม่ใช้ตารางการต่อสู้ แต่จะแบ่งพื้นที่ออกเป็น โซน:

- โซนเดียวกัน (Same Zone) → โจมตีระยะประชิดและระยะไกลได้
- โซนติดกัน (Adjacent Zone) → โจมตีระยะไกลได้ แต่ระยะประชิดไม่ได้
- โซนไกล (Far Zone) → โจมตีระยะไกลได้ แต่จะมีค่าเสียเปรียบ

👉 คุณสามารถเคลื่อนที่ได้ 1 โซนต่อรอบ

การโจมตี (Attacking)

- ทอยโจมตี (Attack Roll) = $d20 + \text{PHYS}$ เทียบกับ ค่าความป้องกัน (AC) ของศัตรู
- ทอยความเสียหาย (Damage Roll) = ลูกเต๋าของอาวุธ (1d6, 1d8, etc.) + ค่าโบนัส
- การโจมตีติดคริติคอล (Natural 20) = ความเสียหายสองเท่า

ตัวอย่างอาชีพ (Example Classes)

- นักรบ (Fighter)
 - HP: 20
 - Attack: $1d6 + \text{PHYS}$
 - Ability: **Second Wind** → รักษา HP 1d8 (1 ครั้ง/การต่อสู้)
- พาลาดิน (Paladin)
 - HP: 20
 - Attack: $1d6 + \text{PHYS}$
 - Ability: **Smite** → เพิ่มความเสียหาย 1d8 (2 ครั้ง/วัน)
 - Ability: **Lay on Hands** → รักษา HP 1d8 (1 ครั้ง/วัน)
- นักบวช (Cleric)
 - HP: 18
 - Attack: $1d6 + \text{PHYS}$
 - Ability: **Pray** (ทอย MENT เทียบกับ DC 13) → เลือกผล:
 - รักษาพันธมิตร (1d8)
 - โจมตีศัตรู (1d8)
 - ขับไล่ผีดิบ
- จอมเวท (Mage)
 - HP: 15
 - Attack: ไม้เท้า (1d6)
 - Spells: 2 ครั้ง/วัน (รูปแบบอิสระ, DC กำหนดโดย DM/LLM)

การพักผ่อนและเวลา (Rest & Time)

- พักสั้น (Short Rest)
 - ระยะเวลา: 1 ชั่วโมง
 - ฟื้นฟู HP 1d8 + ค่าโบนัส
 - ฟื้นฟูความสามารถที่ใช้ได้จำกัด (เช่น Second Wind)
- พักยาว (Long Rest)
 - ระยะเวลา: 8 ชั่วโมง
 - ฟื้นฟู HP ทั้งหมด
 - ฟื้นฟูเวทมนตร์และความสามารถทั้งหมด
 - ต้องทำในที่ที่ปลอดภัย

สถานะผิดปกติ (Conditions)

- หหมดสติ (Unconscious) → HP เหลือ 0, ไม่สามารถทำอะไรก็ได้
- มึนงง (Stunned) → ข้ามรอบของตัวเองในเทิร์นถัดไป
- ถูกพันธนาการ (Restrained) → การทอยเต๋าที่ใช้ค่า PHYS จะเสียเปรียบ
- ตาย (Dead) → ออกจากเกม เว้นแต่จะถูกชุบชีวิต

การเพิ่มระดับ (Leveling Up)

(เป็นทางเลือกสำหรับต้นแบบ)

- +5 HP ต่อเลเวล
- เลือกระหว่าง:
 - +1 ค่าสถานะ
 - เพิ่มการใช้ความสามารถ/เวทมนตร์ใหม่
- ระบบ XP อย่างง่าย: 250 XP ต่อเลเวล

